

Networks and Systems

MM11 — Fault Detection via Hidden Markov Models

Hans-Peter Schwefel | Electronic Systems, 8th Semester

Part 1 Fault Detectors: Definition and Quality Metrics

1.1 What is a Fault Detector?

A **fault detector** is a function $\text{fd}(t)$ that takes system observations $o(t)$ as input (sampled in discrete time as $o(t_i)$) and produces an output representing a conjecture about whether a fault is present. The output can be:

- **Binary:** $\text{fd}(t) = 1$ if a fault is detected, 0 otherwise.
- **Probabilistic:** $\text{fd}(t) \in [0, 1]$, representing an estimated probability of a fault being present.

The **ideal fault detector** outputs exactly 1 whenever the fault is present and 0 otherwise. In practice this ideal is never achieved, and we instead characterise detectors by their error metrics.

1.2 Example: Detection of Node Crashes

A common practical problem is determining whether a remote host is up or down. Two time-out based strategies are used:

- Purpose: Check if another host is up/down.
- Methods – time-out based
 - ▣ Pull: Periodic ping (are you alive?)

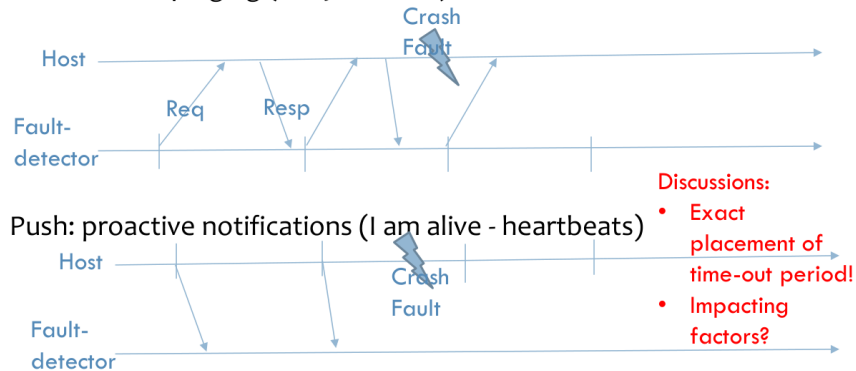


Figure 1: Two approaches to crash detection. **Pull** (top): the fault detector sends periodic ping requests and waits for a response. If no response arrives within the time-out window, a fault is declared. **Push** (bottom): the host sends heartbeat messages proactively; the detector declares a fault if a heartbeat is missed. Key design question: where exactly should the time-out period be placed, and what factors (network delay, jitter, load) affect it?

Design trade-off for the time-out period

A *short* time-out reacts quickly to crashes (low reactivity time) but generates false alarms whenever a reply is merely delayed. A *long* time-out avoids false alarms but slows down crash detection. Choosing the right value requires knowledge of the round-trip time distribution under normal operation.

1.3 True/False Positives and Negatives — The Confusion Matrix

When evaluating any fault detector we need a way to *score* its performance. The fundamental idea is simple: at every point in time the system is either actually faulty or not, and the detector either alarms or it does not. Crossing these two binary facts gives four possible outcomes. The **confusion matrix** is just a table that organises these four cases and asks the question: *how often does each case occur?*

4 Cases (shown as ‘Confusion Matrix’)

(2 system states, S , and 2 fault detector outputs, FD)

	<i>Negatives</i> $FD=0$	<i>Positives</i> $FD = 1$
$S=0$:	$TNR := \Pr(FD=0 \mid S=0)$	$FPR := \Pr(FD=1 \mid S=0)$
$S=1$:	$FNR := \Pr(FD=0 \mid S=1)$	$TPR := \Pr(FD=1 \mid S=1)$

Consequence: $TN=1-FP$; $FN=1-TP$;

Legend: TNR = True Negative Rate, FPR = False Positive Rate, etc.

Figure 2: The four outcomes formed by crossing the true system state S with the fault-detector output FD . Each cell is labelled TP, TN, FP, or FN depending on whether the detector was correct (True) or wrong (False), and whether it alarmed (Positive) or stayed silent (Negative).

To be precise, we work with *rates* (probabilities) rather than raw counts, because the absolute number of events depends on how long we observe the system. With $S \in \{0, 1\}$ denoting the true system state and $FD \in \{0, 1\}$ the detector output, the four conditional probabilities are:

	$FD = 0$ (silent)	$FD = 1$ (alarm)
$S = 0$ (no fault)	$TNR = \Pr(FD = 0 \mid S = 0)$	$FPR = \Pr(FD = 1 \mid S = 0)$
$S = 1$ (fault)	$FNR = \Pr(FD = 0 \mid S = 1)$	$TPR = \Pr(FD = 1 \mid S = 1)$

Where each abbreviation stands for:

- **TNR** — True Negative Rate: the system is healthy and the detector correctly stays silent. This is what we want during normal operation.
- **FPR** — False Positive Rate: the system is healthy but the detector fires anyway. This is a *false alarm* — annoying and potentially costly because operators may react unnecessarily.
- **FNR** — False Negative Rate: the system is faulty but the detector says nothing. This is a *missed fault* — the most dangerous outcome, as a real problem goes unnoticed.
- **TPR** — True Positive Rate (also called *sensitivity* or *recall*): the system is faulty and the detector correctly alarms. This is what we want during a fault.

Because each row of the table describes all outcomes for one system state, each row must sum to 1. This gives the key identities:

$$TNR = 1 - FPR, \quad FNR = 1 - TPR.$$

- **TNR** — True Negative Rate (probability of silence when healthy).

- FPR — False Positive Rate (probability of alarm when healthy).
- FNR — False Negative Rate (probability of silence when faulty).
- TPR — True Positive Rate (probability of alarm when faulty).

This means we only need *one* number per row to describe everything. In practice FPR and TPR are chosen as the two summary metrics because they correspond directly to the two costs we care about: unnecessary alarms and missed faults.

Accuracy summarises the overall correctness of the detector in a single number:

$$\text{Accuracy} = \Pr(\text{FD} = S) = \text{TPR} \cdot \Pr(S = 1) + \text{TNR} \cdot \Pr(S = 0).$$

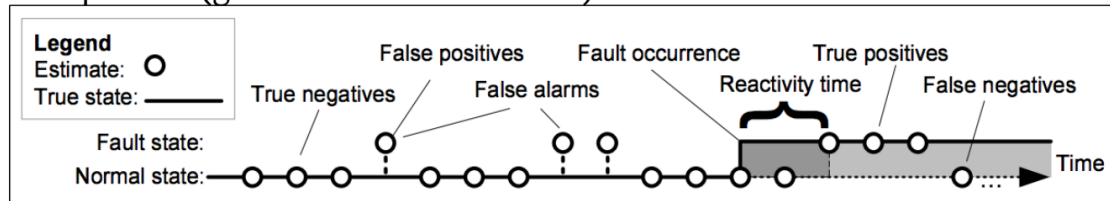
- Accuracy — probability that detector output matches the true system state at any given moment.
- TPR — True Positive Rate (how often we alarm when faulty).
- TNR — True Negative Rate (how often we stay silent when healthy).
- $\Pr(S = 1)$ — prior probability that the system is currently faulty (depends on how reliable the system is overall).
- $\Pr(S = 0)$ — prior probability that the system is currently healthy.

Accuracy can be misleading. Consider a system that is faulty only 1% of the time ($\Pr(S = 1) = 0.01$). A detector that *never* fires achieves 99% accuracy — yet it is completely useless because it never detects anything. This is why FPR and TPR (and the ROC curve, Section 2.2) are preferred over accuracy alone.

1.4 Reactivity and the Full Metrics Picture

8

- Accuracy: probability that fault-detector shows correct result
 - ▣ At moments of estimation versus at any moment in time?
 - ▣ More detailed metrics: Probability of false positives and false negatives
- Reactivity: Distribution of time between fault occurrence and first true positive (given the fault is detected)



From: Nickelsen, A.; Gronbaek, J.; Renier, T.; Schwefel, H.-P.; , "Probabilistic Network Fault-Diagnosis Using Cross-Layer Observations," Advanced Information Networking and Applications, 2009. AINA '09. International Conference on.

→ Calculate these metrics for the heart-beat detector from the previous slide

Figure 3: A timeline showing all four outcomes over time for one fault event. Before the fault: the detector's silence counts as True Negatives (TN), while any spurious alarm counts as a False Positive (FP, also called a false alarm). After the fault occurs, there is a window where the detector has not yet reacted — outputs here are False Negatives (FN). Once the detector finally fires, we enter the True Positive (TP) region. The width of the FN window is the *reactivity time*.

Reactivity is the distribution of the time elapsed between fault occurrence and the *first* true-positive output of the detector (given that the fault is eventually detected). It quantifies how fast the detector responds. A short reactivity is desirable but typically comes at the cost of more false positives.

In other words: FPR and FNR/TPR tell us *how often* the detector is right or wrong; reactivity tells us *how quickly* it notices when something goes wrong. A good detector needs to balance all three.

Part 2 Threshold Detectors and ROC Curves

2.1 Example: Congestion Detection via RTT

A simple but practically important fault detector uses round-trip time (RTT) measurements to decide whether a network path is congested. The idea is straightforward: when a network is congested, packets spend more time queuing in routers, so the RTT grows. If the RTT exceeds some chosen limit, we declare congestion. Formally the detection rule

is:

$$FD(t_i) = \mathbf{1}[RTT_i > \theta],$$

- $FD(t_i)$ — fault detector output at time step t_i ; equals 1 (congestion declared) or 0 (no congestion).
- RTT_i — round-trip time measured at time step i , in milliseconds.
- θ — the detection **threshold**; the single tuning parameter of this detector.
- $\mathbf{1}[\cdot]$ — indicator function; equals 1 if the condition inside is true, 0 otherwise.

The key question is: what should θ be? To answer this we need to look at the RTT distributions under the two possible system states.

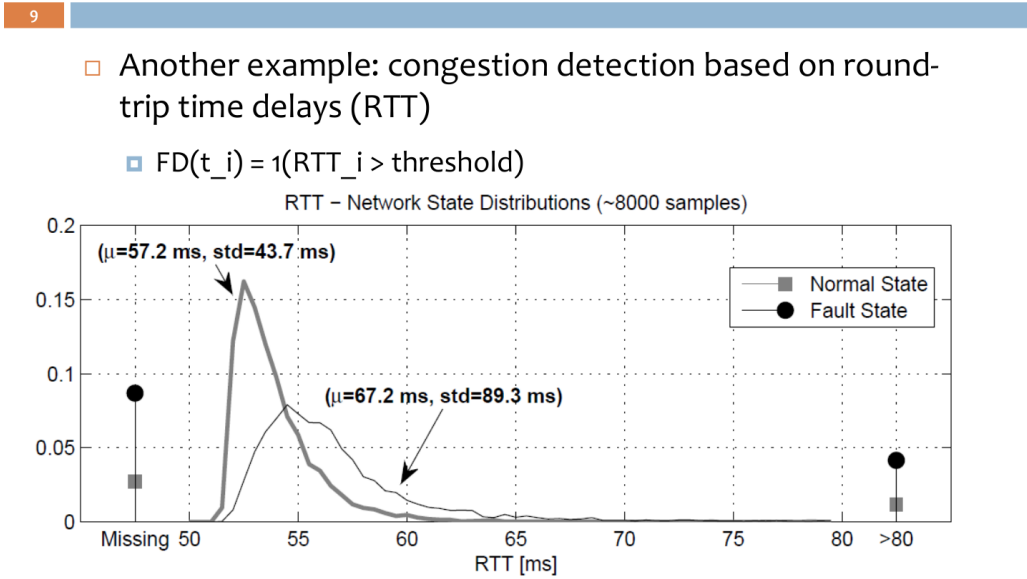
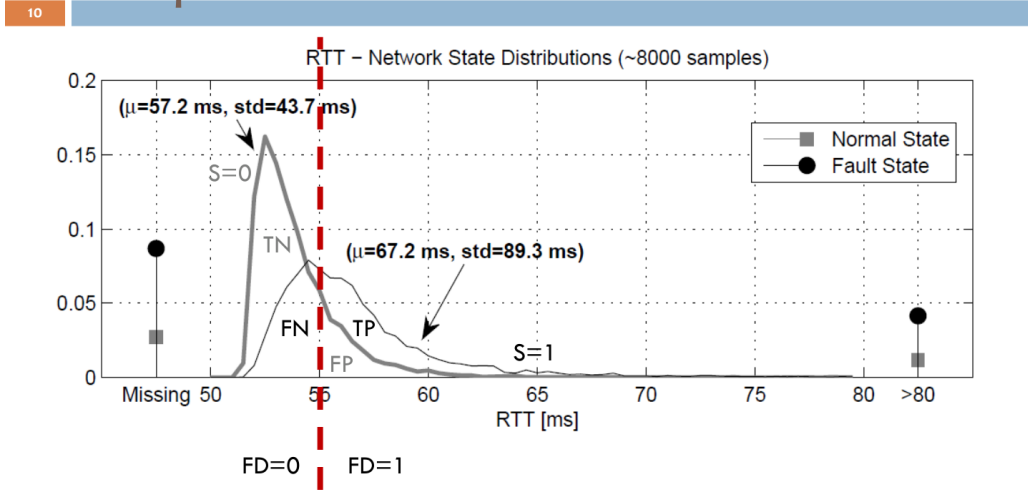


Figure 4: RTT probability distributions measured from approximately 8000 samples, shown separately for the **normal state** (narrow peak, $\mu = 57.2$ ms, $\sigma = 43.7$ ms) and the **fault/congestion state** (much wider and heavier-tailed, $\mu = 67.2$ ms, $\sigma = 89.3$ ms). The x -axis is the RTT value in milliseconds; the y -axis is the probability of observing that RTT value. Notice that the two distributions overlap substantially in the 50–65 ms region, which means no single threshold can perfectly separate them.

The figure shows the core problem: the normal-state distribution and the fault-state distribution are not perfectly separated — they overlap. Every RTT value in the overlap region is ambiguous: it could have come from either state. No matter where we place the threshold θ , we will inevitably misclassify some samples. Moving θ to the left catches more faults (higher TPR) but also flags more healthy samples as congested (higher FPR). Moving θ to the right reduces false alarms (lower FPR) but misses more real faults (lower TPR). This is the fundamental trade-off.



Example: Thresholds = 55ms

Figure 5: Visual derivation of the confusion matrix for a threshold of $\theta = 55$ ms (the vertical red dashed line). Everything to the **left** of the line is labelled $FD = 0$ (no fault declared); everything to the **right** is $FD = 1$ (fault declared). For the $S = 0$ (normal) curve: the area to the left is TN (correctly silent) and the area to the right is FP (false alarm). For the $S = 1$ (fault) curve: the area to the left is FN (missed fault) and the area to the right is TP (correct detection). The relative sizes of these areas give the TNR, FPR, FNR, and TPR for this particular choice of θ .

This figure is the geometric version of the confusion matrix. Instead of abstract probabilities, you can literally see the four regions on the distribution plot. Choosing $\theta = 55$ ms already catches a reasonable portion of the fault distribution on the right, but still leaves some of the normal distribution on the right as false positives.

2.2 ROC Curves

Since there are infinitely many possible values of θ , and each gives a different (FPR, TPR) pair, we need a way to visualise the *full range* of trade-offs available. That is exactly what the **ROC curve** does: it sweeps θ from $-\infty$ to $+\infty$ and plots the resulting (FPR, TPR) pairs as a curve in 2D space. Each point on the curve is one detector configuration; the curve as a whole is the performance landscape.

- E.g. for threshold-based congestion detector, increasing threshold

- ▣ Decreases rate of false positives:

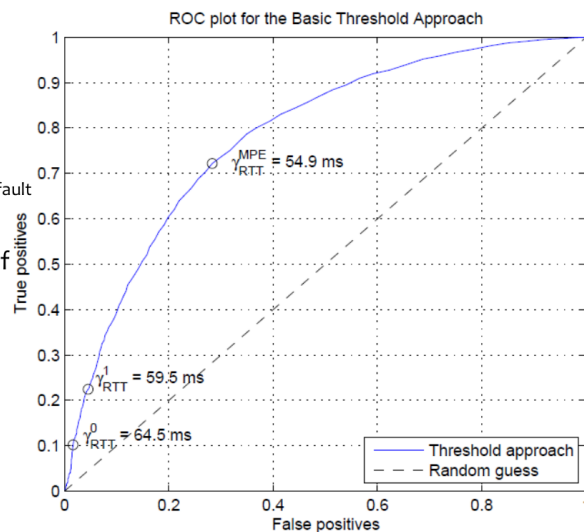
- # false positives / all cases of no fault
 - $\Pr(\text{FP} \mid \text{normal operation})$

- ▣ But also decreases rate of true positives

- # true positives / all cases of faults
 - $\Pr(\text{TP} \mid \text{faulty operation})$

Plotting the resulting curve is called a ROC curve

- Represents behavior of detector for varying parameterization



Discuss meaning of corners of sector and different ROC curve placements

Figure 6: ROC (Receiver Operating Characteristic) curve for the threshold-based congestion detector using RTT measurements. The x -axis is the False Positive Rate (FPR) and the y -axis is the True Positive Rate (TPR). Each labelled point corresponds to a specific threshold value (e.g. $\theta_{\text{RTT}}^0 = 64.5 \text{ ms}$ near the bottom-left). As θ decreases, the operating point moves up and to the right — more faults caught, but more false alarms too. The dashed diagonal represents a random-guess detector ($\text{TPR} = \text{FPR}$).

Reading a ROC curve — what each corner means

- **Top-left corner** (0, 1): the ideal detector. It never false-alarms ($\text{FPR} = 0$) and always catches faults ($\text{TPR} = 1$). Every real detector falls short of this.
- **Bottom-left corner** (0, 0): the detector that always says “no fault.” Zero false alarms, but also zero fault detections. This corresponds to a threshold of $+\infty$.
- **Top-right corner** (1, 1): the detector that always says “fault.” Catches every real fault, but also alarms on every healthy sample. This corresponds to a threshold of $-\infty$.
- **The diagonal** from (0,0) to (1,1): the performance of a completely random detector (e.g. flipping a coin). Any useful detector must lie *above* the diagonal.

A curve that bulges strongly towards the top-left is better than one hugging the diagonal. The ROC curve also lets you pick the threshold that gives the best balance for your specific application (e.g. prefer low FPR for a safety-critical system, or high TPR if missing faults is very costly).

Part 3 Discrete-Time Markov Chains (Revision)

3.1 Definition and Key Properties

A **discrete-time Markov chain (DTMC)** is a probabilistic model for a system that jumps between a set of states at regular time steps. It is the building block we need before defining an HMM.

A DTMC is defined by three ingredients:

- A finite (or countable) **state space** $E = \{1, 2, \dots, N\}$ — the set of all possible conditions the system can be in.
- **Transition probabilities** $p_{jk} = \Pr(X_i = k \mid X_{i-1} = j)$ — the probability of moving from state j to state k in one step. All p_{jk} are collected in the $N \times N$ **transition matrix** $\mathbf{P}$, where row j sums to 1.
- An **initial distribution** $\boldsymbol{\pi}_0$ — a probability vector describing the likelihood of starting in each state.

The fundamental **Markov property** is what makes the model tractable: the state at the next step depends *only* on the current state, not on how we got there. Formally:

$$\Pr(X_i = s \mid X_{i-1} = s_{i-1}, X_{i-2} = s_{i-2}, \dots, X_0 = s_0) = \Pr(X_i = s \mid X_{i-1} = s_{i-1}).$$

- X_i — the random variable representing the system state at step i .
- $s, s_{i-1}, \dots, s_0$ — specific values (realisations) of the state at each time step.
- The left-hand side conditions on the entire history; the right-hand side shows it simplifies to conditioning only on the previous step.

This “memoryless” property means we can propagate the state-probability vector one step at a time using a simple matrix multiplication:

$$\boldsymbol{\pi}_i = \boldsymbol{\pi}_{i-1} \mathbf{P}.$$

- $\boldsymbol{\pi}_i$ — a row vector of length N where entry k gives $\Pr(X_i = k)$, the probability of being in state k at step i .
- $\boldsymbol{\pi}_{i-1}$ — the same vector one step earlier.
- $\mathbf{P}$ — the $N \times N$ transition matrix.

For an **ergodic** chain (one that is irreducible, meaning every state can be reached from every other state, and aperiodic, meaning it does not get stuck in a cycle) the distribution converges to a unique **steady-state** $\boldsymbol{\pi}$ regardless of the starting point:

$$\boldsymbol{\pi} = \lim_{i \rightarrow \infty} \boldsymbol{\pi}_i, \quad \text{satisfying} \quad \boldsymbol{\pi} = \boldsymbol{\pi} \mathbf{P}.$$

- $\boldsymbol{\pi}$ — the steady-state (stationary) distribution; a row vector that is a *left eigenvector* of $\mathbf{P}$ with eigenvalue 1.
- $\boldsymbol{\pi} = \boldsymbol{\pi} \mathbf{P}$ — this equation says “if the distribution is already $\boldsymbol{\pi}$, applying one more step of $\mathbf{P}$ leaves it unchanged.”
- In practice, $\boldsymbol{\pi}$ is found by solving this equation and normalising so that the entries sum to 1.

3.2 Wireless Link Example

As a concrete example, consider a wireless link whose quality fluctuates over time. We model this with three states and assume time steps of one second.

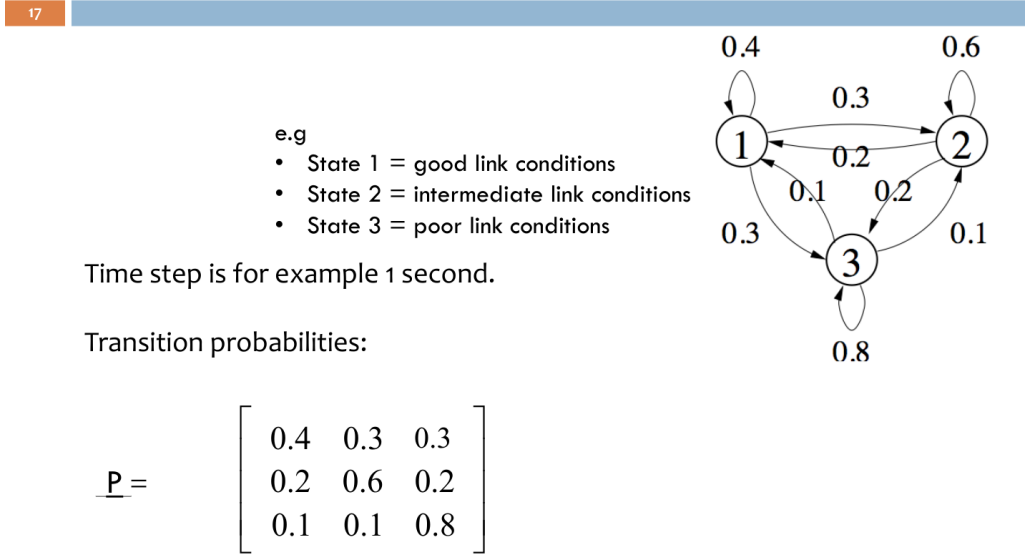


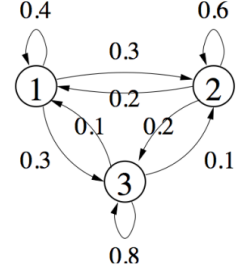
Figure 7: Three-state Markov chain for wireless link quality. **State 1** = good link conditions, **State 2** = intermediate link conditions, **State 3** = poor link conditions. The directed graph shows the transition probabilities on each arrow; self-loops are the probabilities of staying in the same state. The transition matrix $\mathbf{P}$ is shown in the lower-left. Notice that State 3 (poor) has the largest self-loop probability ($p_{33} = 0.8$), meaning poor conditions tend to persist once they occur.

The matrix $\mathbf{P}$ is read row by row: row j gives the probability distribution over the next state, given that the current state is j . For instance, if the link is currently in State 1 (good), there is a 0.4 probability of staying good, 0.3 of moving to intermediate, and 0.3 of deteriorating to poor.

- Given the link is in poor condition right now (i.e., $x_1 = 3$), what is the probability of “poor-poor-good-i/m-i/m-poor” with model M ?

the initial state probability for state i is assumed to be $\pi_0 = [0, 0, 1]$

$$\begin{aligned}
 P(X|\mathcal{M}) &= P(X = \{3, 3, 1, 2, 2, 3\}|\mathcal{M}) \\
 &= P(x_1 = 3) P(x_2 = 3|x_1 = 3) \\
 &\quad \pi_0^3 P(x_3 = 1|x_2 = 3) P(x_4 = 2|x_3 = 1) \\
 &\quad P(x_5 = 2|x_4 = 2) P(x_6 = 3|x_5 = 2) \\
 &= \pi_0(3) p_{33} p_{31} p_{12} p_{22} p_{23} \\
 &= 1 \cdot (0.8)(0.1)(0.3)(0.6)(0.2) \\
 &= 0.00288
 \end{aligned}$$



State 1 = good
State 2 = intermediate (i/m)
State 3 = poor

Figure 8: Worked probability calculation for the specific state sequence $\mathbf{x} = \{3, 3, 1, 2, 2, 3\}$ (poor $\rightarrow$ poor $\rightarrow$ good $\rightarrow$ intermediate $\rightarrow$ intermediate $\rightarrow$ poor), starting from $\pi_0 = [0, 0, 1]$ (certainty of starting in State 3). The probability is computed as a chain of transition probabilities along the path, yielding 0.00288.

The calculation in the figure shows the general rule for computing the probability of any specific state sequence $\mathbf{x} = \{x_1, x_2, \dots, x_T\}$:

$$\Pr(\mathbf{X} = \mathbf{x}) = \pi_0(x_1) \cdot p_{x_1 x_2} \cdot p_{x_2 x_3} \cdots p_{x_{T-1} x_T}.$$

- $\pi_0(x_1)$ — the initial probability of starting in state x_1 . Here $\pi_0(3) = 1$ since we start in State 3 with certainty.
- $p_{x_t x_{t+1}}$ — the transition probability from state x_t to state x_{t+1} ; read from row x_t , column x_{t+1} of $\mathbf{P}$.
- The product runs over all $T - 1$ consecutive state transitions in the sequence.

For the example sequence $\{3, 3, 1, 2, 2, 3\}$:

$$\Pr = 1 \cdot \underbrace{0.8}_{p_{33}} \cdot \underbrace{0.1}_{p_{31}} \cdot \underbrace{0.3}_{p_{12}} \cdot \underbrace{0.6}_{p_{22}} \cdot \underbrace{0.2}_{p_{23}} = 0.00288.$$

Part 4 Hidden Markov Models

4.1 Motivation — Why “Hidden”?

In the wireless link example of Part 3 we assumed we could *directly observe* the link state (good/intermediate/poor). In reality, this is almost never the case. What a device can actually observe is the *outcome of transmissions* — each frame either arrives correctly (C) or with an error (E). The link state drives these outcomes but is itself invisible.

The frame-error rate (FER) depends on the hidden state:

$$\text{FER}(\text{good}) = 10^{-5}, \quad \text{FER}(\text{i/m}) = 0.02, \quad \text{FER}(\text{poor}) = 0.2.$$

- $\text{FER}(s)$ — frame error rate when the link is in state s ; i.e. the probability that any given frame is received in error.
- The values say: in good conditions errors are extremely rare (10^{-5}), in intermediate conditions roughly 1-in-50 frames fail (0.02), and in poor conditions 1-in-5 fail (0.2).

Now consider what we can infer. If we observe the sequence **CCCC** (four correct frames in a row), the link is probably in a good or intermediate state. If we see **EEEE** (four errors in a row), it is almost certainly in a poor state. But what about **CCCE**? The single error could be noise in an otherwise good link, or it could be the start of a deterioration. Deciding this rigorously — and computing an actual probability — is exactly what a **Hidden Markov Model** lets us do.

The word “hidden” refers to the fact that the *state* of the Markov chain is not directly observed; only the *emission* (the symbol produced by that state) is observable.

4.2 Definition of an HMM

□ Main property

- In each state $s \in E$, an ‘observation symbol’ from some alphabet V is generated probabilistically
- The underlying state cannot be observed, only the sequence $\underline{O}=[O_1, O_2, \dots, O_T]$ of generated symbols

□ $\text{HMM} = \langle E, V, \underline{\pi}, \underline{P}, \underline{B} \rangle$

- E : state-space (discrete, finite/infinite), w/o.l.g. $E=\{1,2,\dots,N\}$
- V : set of possible observation symbols (discrete for now), w/o.l.g. $V=\{1,2,\dots,M\}$
- $\underline{\pi}$: initial state probabilities at step 1
- $\underline{P}$: $N \times N$ matrix of state transition probabilities $p_{ij} = \Pr(X_{k+1}=j \mid X_k=i)$
- $\underline{B}$: $N \times M$ matrix of symbol generation probabilities: $b_{ij} = \Pr(O_k=j \mid X_k=i)$

□ Example: 2-state HMM, observations = result from biased coin-toss

- Note: Discrete time Markov model is special case of HMM, namely each column of $\underline{B}$ contains at most one non-zero element
- Exercise: Write a (Matlab) program with input $(\underline{\pi}, \underline{P}, \underline{B}, T)$ that generates a sequence of observations of length T

Figure 9: Formal definition of the HMM tuple and the generative model. At each time step the hidden state X_k first transitions to X_{k+1} according to $\underline{P}$, and then emits an observable symbol O_k according to the row of $\underline{B}$ corresponding to X_k . Only the sequence of emitted symbols $O_1, O_2, \dots, O_T$ is visible to the observer; the state sequence $X_1, X_2, \dots$ is never seen directly.

An HMM is fully described by five components, written as the tuple $\mathcal{M} = \langle E, V, \underline{\pi}, \underline{P}, \underline{B} \rangle$:

HMM components — what each element means:

- $E = \{1, \dots, N\}$ — the **state space**: the finite set of hidden states. In the wireless example $N = 3$ (good, intermediate, poor).
- $V = \{1, \dots, M\}$ — the **observation alphabet**: the finite set of symbols that can be emitted. In the wireless example $M = 2$ (C=correct, E=error).
- π_1 — the **initial state distribution**: a probability vector of length N ; entry i gives the probability of starting in state i at step 1.
- $\mathbf{P}$ — the $N \times N$ **state transition matrix**: entry $p_{ij} = \Pr(X_{k+1} = j \mid X_k = i)$ is the probability of transitioning from state i to state j in one time step. Each row sums to 1.
- $\mathbf{B}$ — the $N \times M$ **emission matrix**: entry $b_{ij} = \Pr(O_k = j \mid X_k = i)$ is the probability of emitting symbol j when in hidden state i . Each row sums to 1.

HMM versus plain Markov chain. A plain DTMC is a special case of an HMM where each state emits exactly one unique symbol (each column of $\mathbf{B}$ has at most one non-zero entry). In that case the state is revealed directly by the emission, so nothing is hidden. The moment a state can emit *multiple different symbols with non-zero probability*, or multiple states can emit the *same* symbol, the state becomes hidden and we need the HMM framework.

Part 5 Efficient Algorithms for HMMs

Three fundamental computational problems arise when working with HMMs. All three take the model $\mathcal{M} = \langle E, V, \pi_1, \mathbf{P}, \mathbf{B} \rangle$ and an observation sequence $\mathbf{o} = [o_1, \dots, o_T]$ as input, but ask different questions about them. Problems 1 and 2 are the focus of this lecture.

5.1 Problem 1: Observation Probability (Forward Algorithm)

Goal: Given model $\mathcal{M}$ and observation sequence $\mathbf{o}$, compute the total probability $\Pr(\mathbf{O} = \mathbf{o} \mid \mathcal{M})$ — i.e. how likely is it that this particular HMM would produce this particular sequence of observations?

This question arises in fault detection when we have *two* trained HMMs — one for normal operation and one for faulty operation — and want to decide which is more consistent with what we currently observe.

A brute-force approach would enumerate every possible hidden state sequence $\mathbf{q} = [q_1, \dots, q_T]$, compute $\Pr(\mathbf{o} \mid \mathbf{q})$ for each, and sum. But there are N^T possible sequences, which grows exponentially. For $N = 3$ states and $T = 100$ observations that is $3^{100} \approx 5 \times 10^{47}$ terms — completely intractable.

The **forward procedure** solves the same problem in $O(N^2T)$ time (quadratic in states, linear in time) by building the answer incrementally, one time step at a time.

The **forward variable** $\alpha_t(i)$ captures everything we need from the past up to step t :

$$\alpha_t(i) := \Pr(O_1 = o_1, O_2 = o_2, \dots, O_t = o_t, X_t = i).$$

- $\alpha_t(i)$ — the joint probability that the first t observations match $o_1, \dots, o_t$ and the hidden state at time t is i .
- $O_1, \dots, O_t$ — the random variables representing observations at steps 1 through t .
- $o_1, \dots, o_t$ — the specific observed values (symbols from V).
- X_t — the hidden state random variable at step t .
- i — a specific state from E .

Algorithm:

1. **Initialise** ($t = 1$):

$$\alpha_1(i) = \pi_1(i) b_{i, o_1} \quad (\text{Matlab: } \alpha_1 = \pi_1 .* \mathbf{B}(:, o_1)')$$

- $\pi_1(i)$ — prior probability of starting in state i .
- b_{i, o_1} — probability of emitting the first observation o_1 from state i (row i , column o_1 of $\mathbf{B}$).

2. **Recurse** ($t = 1, \dots, T - 1$):

$$\alpha_{t+1}(j) = \left(\sum_{i \in E} \alpha_t(i) p_{ij} \right) \cdot b_{j, o_{t+1}} \quad (\text{Matlab: } \alpha_{t+1} = (\alpha_t \mathbf{P}) .* \mathbf{B}(:, o_{t+1})')$$

- $\sum_{i \in E} \alpha_t(i) p_{ij}$ — sum over all states i at step t , weighted by the transition probability p_{ij} to the next state j . This propagates all previous mass to state j .
- $b_{j, o_{t+1}}$ — probability of emitting the next observation o_{t+1} from state j . This gates the propagated mass by how consistent state j is with what was just observed.

3. **Terminate:**

$$\Pr(\mathbf{O} = \mathbf{o}) = \sum_{j \in E} \alpha_T(j).$$

- Sum the forward variable over all states at the final step T . This marginalises out the unknown final state, giving the total observation probability.

- Problem 1: Compute probability of observing a certain sequence $\underline{o}=[o_1, \dots, o_T]$ in a given HMM.

- First (inefficient) approach ('brute-force'):

- Generate all possible state-sequences of length T : $\underline{q}=[q_1, \dots, q_T]$
- Sum up all $\Pr(\underline{o} | \underline{q})$ weighted by $\Pr(\underline{q})$ (total probabilities)
- Problem: Number of paths grows exponentially as N^T

- More efficient (quadratic in N) approach: forward procedure

- Iterative method computing probabilities for pre-fixes of the observation sequence:
 $\underline{\alpha}_t := [\Pr(O_1=o_1, \dots, O_t=o_t, X_t=1), \dots, \Pr(O_1=o_1, \dots, O_t=o_t, X_t=N)]$
- At step $t=1$: $\underline{\alpha}_1(i) = \Pr(O_1=o_1, X_1=i) = \pi_i(i)$ [Matlab Notation: $\underline{\alpha}_1 = \underline{\pi}_1 \cdot \mathbf{B}(:, o_1)'$]
- $t \rightarrow t+1$ ($t=1, 2, \dots, T-1$):

$$\begin{aligned} \underline{\alpha}_{t+1}(i) &= \left(\sum_{j \in E} \underline{\alpha}_t(j) p_{ji} \right) \Pr(O_{t+1}=o_{t+1} | X_{t+1}=i) \\ \underline{\alpha}_{t+1} &= (\underline{\alpha}_t \mathbf{P}) \cdot \mathbf{B}(:, o_{t+1})' \end{aligned}$$

- Finally: $\Pr(\underline{O}=\underline{o}) = \sum_{j \in E} \underline{\alpha}_T(j)$
- Computation can be illustrated in Trellis structure

- Similarly (and identifiers needed later): Backwards procedure

- $\underline{\beta}_t := [\Pr(O_{t+1}=o_{t+1}, \dots, O_T=o_T | X_t=1), \dots, \Pr(O_{t+1}=o_{t+1}, \dots, O_T=o_T | X_t=N)]$
- $\beta_T = 1$ (vector with all elements = 1); $\beta_t = (\mathbf{P} \cdot \mathbf{B}(:, o_{t+1}))' \cdot \beta_{t+1}$

Figure 10: The Forward Algorithm as presented in the slides, including the companion **backward procedure**. The backward variable $\beta_t(i) = \Pr(O_{t+1} = o_{t+1}, \dots, O_T = o_T | X_t = i)$ captures the probability of all *future* observations given that the hidden state at t is i . It is initialised as an all-ones vector at T and propagated backwards to $t = 1$. The backward variable is not needed for Problems 1 or 2 but is required for the Baum-Welch fitting algorithm (Problem 3).

Why dynamic programming works here. The forward variable α_t is a vector of length N that compresses the entire history into just N numbers. At each step we need only N^2 multiplications to update it (one per (i, j) pair in $\mathbf{P}$). Contrast this with the brute-force approach which would need to keep track of all N^t path prefixes explicitly. Dynamic programming discards the path details while retaining the probabilities — the same principle as the Chapman-Kolmogorov recursion for Markov chains.

5.2 Problem 2: Most Likely State Sequence (Viterbi Algorithm)

Goal: Given model $\mathcal{M}$ and observation sequence $\mathbf{o}$, find the specific hidden-state sequence $\mathbf{q}^* = [q_1^*, \dots, q_T^*]$ that is most likely to have generated those observations. Formally, find:

$$\mathbf{q}^* = \arg \max_{\mathbf{q}} \Pr(X_1 = q_1, \dots, X_T = q_T | \mathbf{O} = \mathbf{o}).$$

- $\mathbf{q}^*$ — the optimal (most likely) hidden state sequence.
- $\arg \max_{\mathbf{q}}$ — the argument that maximises the expression; i.e. the state sequence $\mathbf{q}$ achieving the highest probability.
- The conditional is over all N^T possible sequences $\mathbf{q}$, but the Viterbi algorithm finds the maximum without enumerating them all.

For fault detection this is the key question: given what I have observed so far, what is the most likely sequence of system states? If the answer ends in a fault state, we declare a fault.

The structure is identical to the Forward Algorithm but with max replacing $\sum$. However, because we need to reconstruct the best path (not just its probability), we must also store a **backpointer** at every step.

The Viterbi variable $\delta_t(i)$ is the probability of the single best path ending in state i at time t :

$$\delta_t(i) := \max_{q_1, \dots, q_{t-1}} \Pr(X_1 = q_1, \dots, X_{t-1} = q_{t-1}, X_t = i, O_1 = o_1, \dots, O_t = o_t).$$

- $\delta_t(i)$ — the joint probability of the best partial path up to step t that ends in state i , together with all observations $o_1, \dots, o_t$.
- The max is over all possible ways to get to state i at time t via any sequence of earlier states.

The **backpointer** $\Psi_{t+1}(j)$ records which previous state was optimal:

$$\Psi_{t+1}(j) = \arg \max_{i=1, \dots, N} \delta_t(i) p_{ij}.$$

- $\Psi_{t+1}(j)$ — the state at step t that maximises the path probability when followed by state j at step $t + 1$. Stored for every $(t + 1, j)$ pair so the best path can be traced backwards at the end.
- p_{ij} — transition probability from state i to j .

Problem 2: Find 'most likely' state sequence for an observation $\underline{o}=[o_1, \dots, o_T]$ in a given HMM.

- i.e. find the sequence $\underline{q}^*=[q_1^*, \dots, q_T^*]$ that maximizes $\Pr(X_1=q_1, \dots, X_T=q_T \mid \underline{O}=\underline{o})$ (or, equivalently, the joint probability)

- Optimization via pre-fix of length t (Viterbi Algorithm):

$$\delta_t := \left[\max_{q_1, \dots, q_{t-1}} \{ \Pr(X_1=q_1, \dots, X_{t-1}=q_{t-1}, \mathbf{X}_t=\mathbf{1}, O_1=o_1, \dots, O_t=o_t) \}, \right. \\ \left. \dots, \max_{q_1, \dots, q_{t-1}} \{ \Pr(X_1=q_1, \dots, X_{t-1}=q_{t-1}, \mathbf{X}_t=\mathbf{N}, O_1=o_1, \dots, O_t=o_t) \} \right]$$

Algorithm

- $\delta_1 = \underline{\pi}_1 .* \mathbf{B}(:, o_1)'$
- $\delta_{t+1}(j) = [\max_{i=1, \dots, N} \delta_t(i) p_{ij}] B_{j, o_{t+1}}, \Psi_{t+1}(j) = \operatorname{argmax}_{i=1, \dots, N} \delta_t(i) p_{ij}, \quad t=1, 2, \dots, T-1$
- Maximum of probability: $p^* = \max_{i=1, \dots, N} \delta_T(i), \quad q_T^* = \operatorname{argmax}_{i=1, \dots, N} \delta_T(i)$
- state sequence: $q_t^* = \Psi_{t+1}(q_{t+1}^*), \quad t=T-1, \dots, 1$
- Efficient implementations: use of logarithms to avoid multiplications

Figure 11: The Viterbi Algorithm as shown in the slides. The algorithm fills a $N \times T$ grid (the *trellis*) where each cell holds $\delta_t(i)$. At each column only the single best predecessor is kept (via Ψ), discarding all sub-optimal partial paths. After filling the grid, the optimal last state is found from column T , and the full path is recovered by following the backpointers leftward.

Full algorithm:

1. **Initialise** ($t = 1$):

$$\delta_1 = \underline{\pi}_1 .* \mathbf{B}(:, o_1)'$$

Same as the Forward initialisation — the best path of length 1 ending in state i has probability $\pi_1(i) b_{i, o_1}$.

2. **Recurse** ($t = 1, \dots, T-1$):

$$\delta_{t+1}(j) = \left[\max_{i=1}^N \delta_t(i) p_{ij} \right] \cdot B_{j, o_{t+1}}, \quad \Psi_{t+1}(j) = \operatorname{argmax}_{i=1}^N \delta_t(i) p_{ij}.$$

- $\max_i \delta_t(i) p_{ij}$ — among all states i at step t , pick the one that gives the highest path probability when transitioning to state j .
- $B_{j, o_{t+1}}$ — gate by emission probability, just like the Forward recursion.
- $\Psi_{t+1}(j)$ — records which i achieved that maximum, so we can later reconstruct the path.

3. **Terminate**:

$$p^* = \max_{i=1}^N \delta_T(i), \quad q_T^* = \operatorname{argmax}_{i=1}^N \delta_T(i).$$

- p^* — probability of the overall best path.
- q_T^* — the optimal final state (at time T).

4. **Backtrack** ($t = T - 1, \dots, 1$):

$$q_t^* = \Psi_{t+1}(q_{t+1}^*).$$

Follow the stored backpointers from the end of the sequence back to the start to recover the full state path $[q_1^*, \dots, q_T^*]$.

Viterbi vs. Forward — same structure, different question. Both algorithms run in $O(N^2T)$ and initialise identically.

- **Forward** uses $\alpha_{t+1} = (\alpha_t \mathbf{P}) .* \mathbf{B}(\cdot, o_{t+1})$, summing over predecessor states — answers “how probable is this observation sequence?”
- **Viterbi** replaces $\sum$ with $\max$ and stores Ψ — answers “what state sequence most likely produced it?”

In practice both algorithms use logarithms throughout to turn products of small probabilities into sums of log-probabilities, avoiding numerical underflow.

5.3 Problem 3: HMM Parameter Fitting (Baum-Welch)

Goal: Given an observation sequence $\mathbf{o}$ and assumed state space E and observation alphabet V , find the model parameters $\langle \pi_1^*, \mathbf{P}^*, \mathbf{B}^* \rangle$ that maximise $\Pr_{\mathcal{M}}(\mathbf{O} = \mathbf{o})$:

$$\langle \pi_1^*, \mathbf{P}^*, \mathbf{B}^* \rangle = \arg \max_{\pi_1, \mathbf{P}, \mathbf{B}} \Pr_{\langle \pi_1, \mathbf{P}, \mathbf{B} \rangle}(\mathbf{O} = \mathbf{o}).$$

- $\pi_1^*, \mathbf{P}^*, \mathbf{B}^*$ — the optimal initial distribution, transition matrix, and emission matrix respectively, chosen to make the observed data as probable as possible under the model.
- $\Pr_{\mathcal{M}}(\cdot)$ — probability computed with a specific model $\mathcal{M}$; the subscript makes clear that the probability depends on the model parameters being optimised.

This is solved by the **Baum-Welch** algorithm (a special case of the EM algorithm). It iteratively re-estimates all parameters using the forward and backward variables, and is guaranteed to increase the observation likelihood at each iteration (or leave it unchanged at convergence). Convergence to a *local* maximum is the main risk, so multiple random initialisations are used in practice. This algorithm is *not* examinable in this lecture but forms the basis for training HMM-based detectors on real measurement data.

Part 6 HMM-Based Fault Detection

6.1 Single Observation: Congestion Detector

Now we put everything together and use an HMM as a fault detector. The congestion example from Part 2 used a simple threshold on a single RTT measurement. The HMM-based approach is more powerful because it considers a *sequence* of RTT measurements and takes into account that congestion states tend to persist over time (captured by the Markov transition structure).

-
- RTT - Network State Distributions (~8000 samples)
- Legend:
 ■ Normal State
 ● Fault State
- Annotations:
 - Normal State peak: $(\mu=57.2 \text{ ms, std}=43.7 \text{ ms})$
 - Fault State peak: $(\mu=67.2 \text{ ms, std}=89.3 \text{ ms})$
- X-axis: RTT [ms] (Missing, 50, 55, 60, 65, 70, 75, 80, >80)
- Y-axis: Probability Density (0, 0.05, 0.1, 0.15, 0.2)

The key steps to build and apply an HMM-based fault detector:

- Why sequences are more informative than single samples.** A single RTT spike could be a random packet delay in a perfectly healthy network, or it could be the beginning of congestion. The HMM distinguishes between these because the Markov transition structure assigns a higher probability to a state *persisting* than to a state appearing and disappearing within one step. A sustained pattern of high RTTs is therefore much stronger evidence of congestion than a single high RTT.

6.2 Multiple Faults and Observations

A major strength of the HMM framework is that it generalises naturally to scenarios with *multiple simultaneous fault types* and *multiple observable signals*.

27

- Two fault-types:
 - ▣ Poor wireless link
 - ▣ Congestion in core network
- Modeled e.g. by 4-state Markov chain
 - ▣ Measurements on fault-occurrences for state transition probabilities
- 2 observations: RTT distribution and RSSI distribution measured for each state (each combination of faults)
- Discretized distributions mapped to HMM

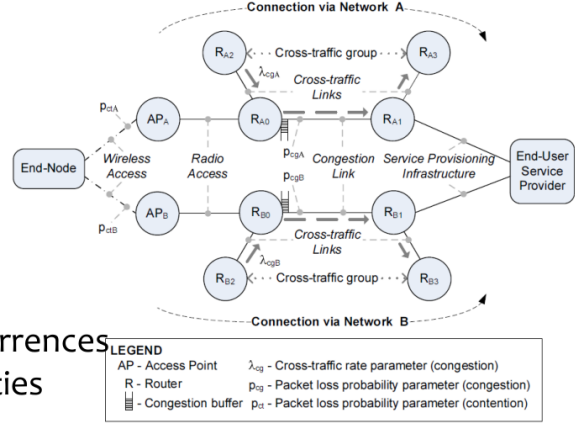


Figure 13: Network topology with two possible fault types occurring independently: **poor wireless link** (between the end-node and the access point) and **congestion in the core network** (in the routers between the access point and the service provider). Because these faults are independent, they can combine into four distinct system states: (no fault, no fault), (wireless fault only), (congestion only), and (both faults). Two observations are available: **RTT** (sensitive to congestion) and **RSSI** (Received Signal Strength Indicator, sensitive to wireless link quality). The joint emission matrix **B** contains the discretised RTT and RSSI distributions for each of the four states, giving the HMM all the information it needs to diagnose which fault (if any) is active.

The two observations (RTT and RSSI) serve complementary roles:

- **RTT** is large when there is congestion in the core network (packets queue in routers), but is unaffected by wireless signal strength.
- **RSSI** (Received Signal Strength Indicator) measures the power of the received radio signal. A low RSSI indicates poor wireless link conditions and correlates with high frame error rates, but RSSI is unaffected by core-network congestion.

By combining both signals in one HMM, the algorithm can distinguish all four system states — something that neither signal alone can do reliably.

Advantages of HMM-based detectors over threshold detectors.

- **Temporal correlations:** a sustained sequence of high RTTs is far more convincing evidence of congestion than a single spike.

- **Multiple simultaneous faults:** handled by adding states to the Markov chain (one state per combination of active faults).
- **Multiple observations:** different signals inform different parts of $\mathbf{B}$, giving cross-layer diagnostic power.
- **Learnable:** if the distributions are unknown, Baum-Welch can estimate $\mathbf{P}$ and $\mathbf{B}$ directly from data.

Part 7 Key Points Summary

Discrete-Time Markov Model elements: a finite state space E ; a transition probability matrix $\mathbf{P}$ (row j gives the distribution over next states from state j); an initial distribution π_0 .

HMM elements: an underlying DTMC $(\pi_1, \mathbf{P})$ describing how the *hidden* state evolves, plus an emission matrix $\mathbf{B}$ linking each hidden state to a probability distribution over an observable alphabet V . The observer sees only the emitted symbols, never the states directly.

Three HMM problems and their algorithms:

1. **Observation probability** — how likely is this observation sequence under this model? Solved by the **Forward algorithm** in $O(N^2T)$.
2. **Most likely state sequence** — what hidden states most plausibly produced these observations? Solved by the **Viterbi algorithm** in $O(N^2T)$.
3. **Parameter fitting** — given data, find the best $\mathbf{P}$ and $\mathbf{B}$. Solved by **Baum-Welch (EM)**; converges to a local optimum.

References

- L. Rabiner, “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition,” *Proc. IEEE*, vol. 77, no. 2, Feb. 1989.
- J. Grønbæk, *Decentralized Fault Management for Service Dependability in Ubiquitous Networks*, PhD Thesis, Aalborg University, Oct. 2010.
- A. Nickelsen, J. Gronbak, T. Renier, and H.-P. Schwefel, “Probabilistic Fault-Diagnosis in Mobile Networks Using Cross-Layer Observations,” in *Proc. AINA 2009*, Bradford, UK, May 2009.
- A. Bondavalli et al., “Threshold-based mechanisms to discriminate transient from in-

termittent faults,” *IEEE Trans. Comput.*, 49(3):230–245, 2000.